Quantitative Finance, 2021
Vol. 21, No. 8, 1309–1323, https://doi.org/10.1080/14697688.2021.1881149

Deep learning-based least squares forward-backward stochastic differential equation solver for high-dimensional derivative pricing

JIAN LIANG, ZHE XU and PETER LI*

Corporate Model Risk, Wells Fargo Bank, San Francisco, CA, USA

(*Received 1 November 2019; accepted 15 January 2021; published online 26 March 2021*)

We propose a new forward-backward stochastic differential equation solver for high-dimensional derivative pricing problems by combining a deep learning solver with a least squares regression technique widely used in the least squares Monte Carlo method for the valuation of American options. Our numerical experiments demonstrate the accuracy of our least squares backward deep neural network solver and its capability to produce accurate prices for complex early exercisable derivatives, such as callable yield notes. Our method can serve as a generic numerical solver for pricing derivatives across various asset groups, in particular, as an accurate means for pricing high-dimensional derivatives with early exercise features.

Keywords: Forward-backward stochastic differential equation (FBSDE); Deep neural network (DNN); Least square regression (LSQ); Bermudan option; Callable yield note (CYN); High-dimensional derivative pricing

JEL classifications: C45, C63

1. Introduction

As explained in the well-known book by Hull (2018), a financial derivative, or simply a derivative, can be defined as a financial instrument whose value depends on (or derives from) the values of other, more basic underlying variables. The underlying variables can be traded assets such as common stocks, commodities, bonds, etc. They can also be stock indexes, interest rates, foreign exchange rates, etc. Derivatives can be used by hedgers to reduce the risk that they face from potential future movements in a market variable and by speculators to bet on the future direction of a market variable. The most common derivative types are futures contracts, forward contracts, swaps and options.

Derivative pricing has been widely studied in academia and industry. Numerical methods have to be used for derivative pricing except for simple derivatives, such as futures, forwards, swaps, and European vanilla options. Tree, PDE and Monte Carlo are the three major methods in pricing complex derivatives. However, both the tree approach and the classical finite difference based PDE approach are infeasible for high-dimensional derivative pricing due to implementation complexity and the numerical burden. This is the well-known 'curse of dimensionality'. Therefore, the Monte Carlo method is widely used in high-dimensional derivative pricing. When pricing early exercisable products, e.g. American options, Bermudan options, callable structured notes, etc. some additional numerical procedures have to be embedded in the Monte Carlo method in order to determine the optimal exercise strategy. Since it is computationally impractical to perform a sub-MC simulation at the early exercise time to compute the expected payoff from continuation, various techniques have been proposed. Barraquand and Martineau (1995) proposed a stratified state method which sorts the stock price paths according to a state variable (rather than the stock price) to determine the payoff. However, in Barraquand and Martineau's method, an error estimate of the results cannot be obtained. Broadie and Glasserman (1997) proposed a simulated tree method to price American options, and their approach can also generate upper and lower bounds for those options. Longstaff and Schwartz (2001) proposed a least squares Monte Carlo algorithm to price American options, and in their approach a least squares regression was introduced in the early exercise step to estimate the expected payoff from continuation. Since it is computationally efficient and easily implemented (Stentoft 2004), least squares Monte Carlo is the most widely used algorithm among practitioners

*Corresponding author. peter.li@wellsfargo.com

for pricing high-dimensional derivatives with early exercise features.

The application of machine learning in derivative pricing can be traced back to as early as the 1990s, when Hutchinson *et al*. (1994) used a neural network in a nonparametric regression to estimate option prices. The application of neural networks combined with regression to tackle early exercise options, such as American options pricing problems, has been reported by Kohler *et al.* (2010). In Kohler *et al.*'s work, the neural network was used as an optimization tool for non-parametric regression. In a recent study (De Spiegeleer *et al.* 2018) applied machine learning in Gaussian Process Regression to predict option prices from the neural network with estimated product, market and model parameters. More recently, Becker *et al.* (2019) developed a deep neural network based method to solve the optimal stopping problem recursively over each allowed stopping time. Later, Becker *et al.* (2019) extended the method to solve the problem in a single step. The authors used a neural network to approximate the optimal stopping time, and demonstrated the application of their method in Bermudan options with various types of payoff. As well as using machine learning as regression tools or to 'learning' optimal stopping times, researchers have used machine learning techniques to approximate the solutions to parabolic PDEs associated with derivative pricing, in particular, for high-dimensional PDEs where classical approaches are challenged. Sirignano and Spiliopoulos (2017, 2018) applied the deep Galerkin method to solve the high-dimensional PDEs that arise in quantitative finance applications, including option pricing.

E *et al.* (2017) and Han *et al.* (2018) proposed an innovative algorithm, referred to as a forward DNN, in which deep neural network is used to solve non-linear parabolic PDEs. Through generalized Feynman-Kac theorems they represented the PDE in terms of equivalent backward stochastic differential equations (BSDEs), then developed a deep neural network algorithm to solve the BSDEs. Their algorithm is straightforward to implement and can be directly applied to European style high-dimensional derivative pricing. Raissi (2018) proposed a different loss function from that in the work of E *et al.* (2017) and applied the neural network directly to the solution of interest. As a result, in Raissi's algorithm, the solution covers the whole space-time domain, not just the initial point as is done in Weinan E's algorithm. In addition, Raissi's algorithm enjoys the merit of an independence between the number of parameters of the neural network and fineness of the time discretization. Note that Weinan E's or Raissi's method is more appropriate for European style derivative pricing, than for derivatives with early exercise features. Fujii *et al.* (2019) demonstrated that the use of asymptotic expansion as prior knowledge in the forward DNN method could drastically reduce the loss function and accelerate the convergence speed. They also extended the forward DNN method for reflected BSDEs (El Karoui *et al.* 1997) which can be used for American basket option pricing. Though the reflected BSDE approach to American and Bermudan option valuation has attracted much interest in academia, for instance, El Karoui *et al.* (1997), Bouchard and Chassagneux (2008), to the best of our knowledge it has not gained popularity in the financial industry, primarily due to the fact that industry practitioners mainly rely on the standard PDE approach and least squares Monte Carlo simulation for American and Bermudan options. Wang *et al.* (2018) proposed a backward DNN algorithm for pricing Bermudan swaptions with a LIBOR market model. However, the validity and accuracy of their backward DNN for Bermudan swaptions is not clear since there are no numerical studies in the work of Wang *et al.* to compare the results from the backward DNN to those from the classical approaches such as the least squares Monte Carlo simulation. Since the discounted payoff on a simulation path is taken as the continuation value conditional on no early exercise, the price of a Bermudan swaption is expected to be biased in Wang *et al.*'s algorithm. Huré *et al.* (2020) introduced a backward scheme for high-dimensional nonlinear PDEs, where the loss functions are minimized at each time step. This differs from our global minimization involving hyperparameters, and their proposed scheme is not able to accurately represent a solution to a very complex structure in high dimensions.

In this paper, we propose a backward deep learning-based least squares forward-backward stochastic differential equation solver for pricing high-dimensional derivatives, in particular, with early exercise features. In our work, the neural network is used to solve the BSDE. In comparison to Wang *et al.*'s algorithm, which is only applicable to a vanishing drift term, our algorithm can be used for general drift functions. In addition, in our algorithm the least squares regression is used to determine the optimal early exercise. Even though there has been much research on using neural networks to approximate the solutions of PDEs for the purpose of derivative pricing, very few studies have been reported to assess its efficiency in comparison with classical numerical methods. Our work also aims at closing this gap by comparing the DNN based algorithms with classical Monte Carlo simulation and hence providing guidance on which situations DNN based algorithms are more efficient.

The rest of this paper is organized as follows. In Section 2, we introduce some basic background knowledge for forward-backward stochastic differential equations (FBSDEs), which is the key knowledge to our least square backward DNN method. In addition, we briefly explain the concept of Bermudan options and callable yield notes, which will be used as examples in our numerical testing. The forward DNN method (E *et al.* 2017) is described in Section 3. In Section 4, we first outline the backward DNN method, and then introduce the least squares backward DNN method. Numerical results for Bermudan options and callable yield notes are presented in Section 5. Section 5 also includes efficiency tests of the DNN based methods. We conclude our paper in Section 6.

2. Background knowledge

In this section, we first introduce some basics of forward-backward stochastic differential equations (FBSDEs) and then describe the contract characteristics of Bermudan options and callable yield notes (CYN). Both instruments are used to perform our numerical tests in Section 5.

2.1. Forward-backward stochastic differential equation

Many derivative pricing and continuous-time portfolio optimization problems in financial mathematics can be reformulated in terms of backward stochastic differential equations (BSDEs). El Karoui *et al.* (1997) is a good systematic reference for using BSDEs in finance. These equations are non-anticipatory terminal value problems for stochastic differential equations (SDEs) of the form

$$\begin{aligned} -\mathrm{d}Y_t &= f\left(t, Y_t, Z_t\right) \mathrm{d}t - Z_t \,\mathrm{d}W_t, \\ Y_T &= \xi, \end{aligned} \tag{1}$$

where W_t is a standard d-dimensional Brownian motion defined on a complete probability space. The square-integrable terminal condition ξ (measurable with respect to filtration generated up to time T by the Brownian motion) and the so-called generator f are given.

When the BSDEs are used in derivative pricing, Y_t corresponds to the derivative value and Z_t is related to the hedging portfolio as shown in Equation (5) below. Both the derivative value and the hedging portfolio are functions of time t and asset value X_t, i.e. $Y_t = Y(t, X_t)$ and $Z_t = Z(t, X_t)$. In many portfolio optimization problems, Y_t corresponds to the value process, while an optimal control can often be derived from Z_t. Finally, BSDEs can also be applied in order to obtain Feynman-Kac type representation formulas for nonlinear parabolic PDEs. In the equation above, Y_t and Z_t correspond to the solution and the gradient of the PDE, respectively.

In this paper, we will focus on a forward-backward stochastic differential equation (FBSDE) of the form

$$\begin{aligned} \mathrm{d}X_t &= \mu\left(t, X_t\right) \mathrm{d}t + \sigma\left(t, X_t\right) \mathrm{d}W_t, \\ X_0 &= x, \\ -\mathrm{d}Y_t &= f\left(t, X_t, Y_t, Z_t\right) \mathrm{d}t - Z_t \,\mathrm{d}W_t, \\ Y_T &= g\left(X_T\right). \end{aligned} \tag{2}$$

Here $g(\cdot)$ is the payoff function. The name forward-backward comes from the fact that X moves forward as its initial value is given, Y moves backward as its terminal value is given. Suppose X_t is the stock value and it follows

$$\mathrm{d}X_t = (r - q) X_t \,\mathrm{d}t + \sigma X_t \,\mathrm{d}W_t. \tag{3}$$

For simplicity, we assume r is the constant discount rate, q is the constant dividend rate, and σ is the constant volatility. We only use subscript when it is necessary. Proceeding in the same fashion as in the derivation of the Black-Scholes PDE, we construct a portfolio $\Pi = Y - \Delta X$, and Δ will be selected so that the value of the portfolio is deterministic.

$$\begin{aligned} \mathrm{d}\Pi &= \mathrm{d}Y - \Delta \,\mathrm{d}X - q\Delta X \,\mathrm{d}t \\ &= \left(\frac{\partial Y}{\partial t} + \frac{1}{2}\sigma^2 X^2 \frac{\partial^2 Y}{\partial X^2}\right) \mathrm{d}t + \frac{\partial Y}{\partial X} \,\mathrm{d}X - \Delta \,\mathrm{d}X - q\Delta X \,\mathrm{d}t \end{aligned}$$

The term $q\Delta X dt$ arises since the stock pays dividend which decreases the value of the portfolio by the amount of the dividend. If we select $\Delta = \frac{\partial Y}{\partial X}$, we then have

$$\mathrm{d}\Pi = \left(\frac{\partial Y}{\partial t} + \frac{1}{2}\sigma^2 X^2 \frac{\partial^2 Y}{\partial X^2}\right) \mathrm{d}t - q\Delta X \,\mathrm{d}t.$$

Since the value of the portfolio is risk free, we must have

$$\mathrm{d}\Pi = r\Pi \,\mathrm{d}t = r\left(Y - \Delta X\right) \mathrm{d}t.$$

This leads to the following Black Scholes PDE

$$\frac{\partial Y}{\partial t} + \frac{1}{2}\sigma^2 X^2 \frac{\partial^2 Y}{\partial X^2} + (r - q) X \frac{\partial Y}{\partial X} - rY = 0. \tag{4}$$

From Itô's Lemma, we have

$$\begin{aligned} \mathrm{d}Y &= \left(\frac{\partial Y}{\partial t} + \frac{1}{2}\sigma^2 X^2 \frac{\partial^2 Y}{\partial X^2}\right) \mathrm{d}t + \frac{\partial Y}{\partial X} \,\mathrm{d}X \\ &= \left(rY - (r - q) X \frac{\partial Y}{\partial X}\right) \mathrm{d}t \\ &\quad + \frac{\partial Y}{\partial X} \left((r - q) X \,\mathrm{d}t + \sigma X \,\mathrm{d}W\right) \\ &= rY \,\mathrm{d}t + \sigma X \frac{\partial Y}{\partial X} \,\mathrm{d}W, \end{aligned}$$

or

$$-\mathrm{d}Y = -rY \,\mathrm{d}t - \sigma X \frac{\partial Y}{\partial X} \,\mathrm{d}W, \tag{5}$$

that is $f = -rY$, and $Z = \sigma X \frac{\partial Y}{\partial X}$ in Equation (2).

The above statement can be easily extended to a high-dimensional derivative pricing ($Y = Y(t, X^1, X^2, \ldots, X^d)$), and we have (neglecting subscript t)

$$\begin{aligned} \mathrm{d}X^i &= \mu^i\left(t, X^i\right) \mathrm{d}t + \sigma^i\left(t, X^i\right) \mathrm{d}W^i \\ X_0^i &= x^i \\ -\mathrm{d}Y &= -rY \,\mathrm{d}t - \sum \sigma^i X^i \frac{\partial Y}{\partial X^i} \,\mathrm{d}W^i \\ Y_T &= g\left(X_T^1, X_T^2, \ldots, X_T^d\right) \\ cov\left(\mathrm{d}W^i, \mathrm{d}W^j\right) &= \rho^{ij} \,\mathrm{d}t \quad |\rho^{ij}| < 1. \end{aligned} \tag{6}$$

Here, $\{\rho^{ij}\}$ is the correlation matrix of the Brownian motions, and it is symmetric, positive semi-definite.

2.2. Bermudan options

A Bermudan option is a type of an exotic option such that the option holder can only exercise on predetermined dates. The Bermudan option is exercisable on the date of expiration, and on certain specified dates that occur between the purchase date and the date of expiration. A Bermudan option can be considered as a hybrid of an American option (exercisable on any dates before and including expiration) and a European option (exercisable only at expiration). The payoff function (or intrinsic value) of a Bermudan call at time t if not exercised

early is given by

$$U\left(t, X_t\right) = \max\left(\sum_{i=1}^{d} \omega_i X^i\left(t\right) - K, 0\right), \tag{7}$$

where K is the strike of the option and the weights ω_i are given constants. When an exercise event happens, the option expires and the holder will receive its intrinsic value. Given the exercise times as $0 < t_1 < t_2 < \cdots . < t_n \leq T$, the value of a Bermudan option at time t can be written as

$$V(t, X_t) = \sup_{\tau \in \mathcal{T}(t)} \mathbb{E}^{Q}\left[D(t,\tau) U\left(\tau, X_\tau\right) | \mathcal{F}_t\right], \tag{8}$$

where $D(t,\tau)$ is the discount factor, $\mathcal{T}(t) = \{t_k | t_k > t\}$ is the set of exercise times, and the expectation is taken under risk neutral measure. As the option holder has the right to early exercise, the value of a Bermudan option is the supremum of all the possible early-exercised values.

2.3. *Callable yield notes*

Callable yield note (CYN), also called worst of issuer callable, is a yield enhancement product. The performance of a CYN is capped by a coupon that is guaranteed by an issuer. As the name implies, the issuer, at his discretion, can call the product, usually on predefined early exercise dates. The early issuer exercise dates usually coinside with the coupon dates. The underlying entities are generally composed of several stocks or stock indices; thus making it a product based on a worst-of function. The call notice dates for a CYN are often identical to the coupon record dates. We denote the coupon record dates as t_i, $i = 1, 2, \ldots, n$, with $t_n = T$ being equal to the expiry date T. The coupon payments are subject to a barrier condition and the knock-in barrier is observed at expiry. The coupon payments per unit of notional are

$$\begin{aligned} c\left(t_i\right) &= r_i \Theta\left(p\left(t_i\right) - B_i\right) \quad \text{for } i = 1, 2, \ldots, N-1 \\ c\left(t_n\right) &= r_n \Theta\left(p\left(T\right) - B_n\right) \\ &\quad - \Theta\left(B - p\left(T\right)\right) \max\left(K - p\left(T\right), 0\right), \end{aligned} \tag{9}$$

where r_i is the contingent coupon with coupon barrier B_i on ith coupon date, B is the knock-in barrier at expiry, K is the knock-in put strike, and $p(t)$ is the relevant performance since trade inception. $p(t)$ is defined as

$$p\left(t\right) = \min_{j \in \{1,2,\ldots,d\}} \left[\frac{X^j\left(t\right)}{X^j\left(0\right)}\right], \tag{10}$$

and $\Theta(x)$ is the Heaviside function

$$\Theta\left(x\right) = \begin{cases} 1 & \text{for } x \geq 0 \\ 0 & \text{otherwise.} \end{cases} \tag{11}$$

Furthermore, upon redemption (at the scheduled expiry or early issuer exercise date) the principal notional is returned to the holder. The payoff function of a CYN at time t if not exercised early is given by

$$U(t, X_t) = \text{notional} + \text{notional} \sum_{t_i \leq t} \frac{c(t_i)}{D(t_i, t)}, \tag{12}$$

where $D(t_i, t)$ is the discount factor. Given the early issuer exercise times as $0 < t_1 < t_2 < \cdots . < t_n \leq T$, the value of a callable yield note at time t is

$$V(t, X_t) = \inf_{\tau \in \mathcal{T}(t)} \mathbb{E}^{Q}\left[D(t,\tau) U\left(\tau, X_\tau\right) | \mathcal{F}_t\right], \tag{13}$$

where $D(t,\tau)$ is the discount factor, $\mathcal{T}(t) = \{t_k \,|\, t_k > t\}$ is the set of early issuer exerecise times, and the expectation is taken under risk neutral measure. As the CYN issuer has the right to early exercise, the value of a CYN is the infimum of all the possible early-exercised values.

3. Forward DNN method

Forward solvers using deep neural network (DNN) have been developed mainly by E *et al.* (2017) and Han *et al.* (2018). FBSDEs (Equation (2)) can be numerically solved in the following way:

- Simulate sample paths for the FBSDE using a standard Monte Carlo method.
- Approximate $z = \frac{\partial Y}{\partial X}$ using a deep neural network (DNN), then plug into the FBSDE to propagate forward in time.

Since the BSDE (the SDE for Y) in Equation (2) is solved forward in time, we call this approach the 'forward DNN' to distinguish it from the backward DNN method (explained below in Section 4.1) where Y is solved backward in time. We briefly describe the forward DNN method in this section. More details can be found in E *et al.* (2017) and Han *et al.* (2018). For simplicity, we use 1D (single underlier) case as an example. The high-dimensional case is similar. Specifically, we consider a time discretization $\pi = \{t_0, \ldots, t_N\}$ of the interval $[0, T]$, i.e. $0 = t_0 < t_1 < \cdots < t_N = T$, where we assume valuation date $= 0$ and expiration date$= T$. Denoting $h_i = t_{i+1} - t_i$ and $dW_i = W_{t_{i+1}} - W_{t_i}$.

(i) M Monte Carlo (MC) paths $\{X_i^{(j)} : i = 0, 1, \ldots, N; j = 1, 2, \ldots, M; X_0^{(j)} \equiv X_0\}$ of the underlying stock (X_i short for X_{t_i}, similarly for other notations) are sampled by an Euler scheme through

$$X_{i+1}^{(j)} = X_i^{(j)} + \mu\left(t_i, X_i^{(j)}\right) h_i + \sigma\left(t_i, X_i^{(j)}\right) dW_i^{(j)}. \tag{14}$$

This step is the same as the standard Monte Carlo pricer. Other discretization schemes can be used, for instance, log-Euler discretization or Milstein discretization (Mil'shtejn 1975).

(ii) At time $t_0 = 0$, Y_0 and $z_0 = \frac{\partial Y}{\partial X}(0, X_0)$ are randomly selected, and we set $Z_0 = \sigma(0, X_0) X_0 z_0$. Both Y_0 and z_0 are uniform random numbers with the range empirically chosen.

(iii) For $t_i \in \pi$, we have

$$Y_i^{(j)} - Y_{i+1}^{(j)} = f(t_i, X_i^{(j)}, Y_i^{(j)}, Z_i^{(j)})h_i - Z_i^{(j)} dW_i^{(j)}, \tag{15}$$

or

$$Y_{i+1}^{(j)} = Y_i^{(j)} - f(t_i, X_i^{(j)}, Y_i^{(j)}, Z_i^{(j)})h_i + Z_i^{(j)} dW_i^{(j)}. \tag{16}$$

At each time step t_i, given $\{X_i^{(j)} : j = 1, 2, \ldots, M\}$, a deep neural network (DNN) approximation is used for $\{z_i^{(j)} : j = 1, 2, \ldots, M\}$. Notice that one DNN is used for all MC paths at each time $t = t_i$. For our one dimensional problem, we introduce a DNN $\theta_i : \mathbb{R} \to \mathbb{R}$ of the form

$$\theta_i = a_K^i \circ a_{K-1}^i \circ \cdots a_1^i \circ, \tag{17}$$

where $a_1^i : \mathbb{R} \to \mathbb{R}^{d_1}$, $a_2^i : \mathbb{R}^{d_1} \to \mathbb{R}^{d_2}$, $\cdots$, $a_K^i : \mathbb{R}^{d_{K-1}} \to \mathbb{R}$ are affine functions, and d_k $(k = 1, 2, \ldots, K-1)$ are integers. $\{z_i^{(j)} = \theta_i(X_i^{(j)}) : j = 1, 2, \ldots, M\}$, and $Z_i^{(j)} = \sigma(t_i, X_i^{(j)}) X_i^{(j)} z_i^{(j)}$. Then, the FBSDE propagates forward in time direction from t_i to t_{i+1} using Equation (16). Along each Monte Carlo path, as the FBSDE propagates forward from time 0 to T, $Y_N^{(j)}$ is estimated as $Y_N^{(j)}(Y_0, z_0, \theta)$, where $\theta = (\theta_1, \ldots, \theta_{N-1})$ are all hyper-parameters for the $N-1$ neural networks.

(iv) A natural loss function will be

$$L_{Forward} = \frac{1}{M} \sum_{j=1}^{M} \left(Y_N^{(j)}(Y_0, z_0, \theta) - g\left(X_N^{(j)}\right) \right)^2. \tag{18}$$

Here $g(\cdot)$ is the payoff function.

(v) The Adam optimization (in TensorFlow library) is used to minimize the loss function

$$\left(\widetilde{Y}_0, \widetilde{z}_0, \widetilde{\theta}\right) = \underset{(Y_0, z_0, \theta)}{\operatorname{argmin}} \frac{1}{M} \sum_{j=1}^{M} \left(Y_N^{(j)}(Y_0, z_0, \theta) - g\left(X_N^{(j)}\right) \right)^2. \tag{19}$$

The estimated $\widetilde{Y}_0$ is the desired derivative value at $t = 0$. More details about the use of Adam optimization to solve the above minimization problem can be found in E *et al.* (2017), Han *et al.* (2018).

4. Least square backward DNN method

Since the forward DNN method above can not be applied to price options with early exercise features, such as Bermudan options, Wang *et al.* (2018) proposed a backward DNN method to price Bermudan swaptions under LIBOR market model. As discussed in the introduction of this paper, Wang *et al.*'s algorithm is only applicable to a backward process with a vanishing drift term ($f = 0$ in Equation (2)) and the Bermudan swaption price can be biased since the discounted payoff along a simulation path is taken as the continuation value conditional on not exercised early. Different from their algorithm, our backward DNN method can be applied to general drift functions. We also apply the least square regression to estimate continuation value in order to determine the optimal exercise decision.

4.1. Backward DNN method

We would like to propagate backward in time direction and apply the call/put and coupon events to the derivative value. From Equation (16), we have

$$Y_i^{(j)} = Y_{i+1}^{(j)} + f\left(t_i, X_i^{(j)}, Y_i^{(j)}, Z_i^{(j)}\right) h_i - Z_i^{(j)} \, \mathrm{d}W_i^{(j)}. \tag{20}$$

As we propagate backward in time direction from t_{i+1} to t_i, $Y_{i+1}^{(j)}$ is known while $Y_i^{(j)}$ is to be determined. We use 1st order Taylor expansion to do the approximation.

$$\begin{aligned} Y_i^{(j)} \approx Y_{i+1}^{(j)} &+ \left(f\left(t_i, X_i^{(j)}, Y_{i+1}, Z_i^{(j)}\right) \right. \\ &\left. - \frac{\partial f}{\partial Y}\left(t_i, X_i^{(j)}, Y_{i+1}^{(j)}, Z_i^{(j)}\right)\left(Y_{i+1}^{(j)} - Y_i^{(j)}\right) \right) h_i \\ &- Z_i^{(j)} \, \mathrm{d}W_i^{(j)}, \end{aligned} \tag{21}$$

which leads to

$$\begin{aligned} Y_i^{(j)} \approx Y_{i+1}^{(j)} &+ \frac{1}{1 - \dfrac{\partial f}{\partial Y}\left(t_i, X_i^{(j)}, Y_{i+1}^{(j)}, Z_i^{(j)}\right) h_i} \\ &\times \left(f\left(t_i, X_i^{(j)}, Y_{i+1}^{(j)}, Z_i^{(j)}\right) h_i - Z_i^{(j)} dW_i^{(j)} \right). \end{aligned} \tag{22}$$

One can use higher order Taylor expansion to achieve more accurate approximation. For our particular equations (Equation (5)), 1st order Taylor expansion approximation is indeed the exact solution. And we have

$$Y_i^{(j)} = \frac{Y_{i+1}^{(j)} - Z_i^{(j)} \, \mathrm{d}W_i^{(j)}}{1 + rh_i}. \tag{23}$$

Starting from $t_N = T$, we can propagate backward in time direction to $t_0 = 0$, and obtain the estimated initial value $Y_0^{(j)}(\theta)$ for each sampled path, where $\theta = (\theta_1, \ldots, \theta_{N-1})$ are all hyper-parameters for the $N-1$ neural networks. The ideal case will be that the estimated initial values $Y_0^{(j)}(\theta)$ concentrate to one point. Therefore, the loss function is defined as

$$L_{Backward} = \frac{1}{M} \sum_{j=1}^{M} \left(Y_0^{(j)}(\theta) - \frac{1}{M} \sum_{j=1}^{M} \left(Y_0^{(j)}(\theta) \right) \right)^2. \tag{24}$$

This implies that we are trying to minimize the variance of the estimated initial values. The Adam optimization is used to

minimize the loss function $L_{Backward}$ and estimate Y_0 as

$$\widetilde{Y}_0 = \frac{1}{M}\sum_{j=1}^{M}\left(Y_0^{(j)}(\widetilde{\theta})\right), \tag{25}$$

where

$$\widetilde{\theta} = \underset{\theta}{\operatorname{argmin}} L_{Backward}. \tag{26}$$

Finally the estimated $\widetilde{Y}_0$ is our desired derivative value at $t = 0$.

4.2. Least square regression

We use a Bermudan call option to explain how the conditional expectation of the payoff estimated from least square regression is used to determine optimal strategy at an early exercise time. The readers are referred to the classical paper by Longstaff and Schwartz (2001) for more details.

Without loss of generality, we assume the exercise time as $t_k \in \{t_1, t_2, \ldots, t_n\}$, which is a subset of the time discretization $\pi = \{t_0 = 0, \ldots, t_N = T\}$. We use 1D (single underlier) case as an example. The high-dimensional case follows straightforwardly. Following the classical optimal stopping theory (such as Chapter 6 of Neveu (1975) and its application in American options valuation with a least squares regression by Clément *et al.* (2002)), we can define the Snell envelope $(V_k)_{k=1,\ldots,n}$ of the payoff (Equation (7)) by

$$V_k = \sup_{\tau \in \mathcal{T}(t_k)} \mathbb{E}^Q[D(t_k,\tau)U(\tau, X_\tau)|\mathcal{F}_k], \quad k = 1, 2, \ldots, n. \tag{27}$$

The dynamic programming principle can be constructed as (X_i is shorthand for X_{t_i}):

$$\begin{cases} V_N = U(t_N, X_N) \\ V_k = \mathbb{E}^Q[D(t_k, t_{k+1})V_{k+1}|\mathcal{F}_k] & \text{if } t_k \text{ is not exercise time} \\ V_k = \max\left(U(t_k, X_k), \right. \\ \qquad \left. \mathbb{E}^Q[D(t_k, t_{k+1})V_{k+1}|\mathcal{F}_k]\right) & \text{if } t_k \text{ is exercise time} \end{cases} \tag{28}$$

Here V_k is the Bermudan option value at time t_k, or $V_k = V(t_k, X_k)$ in Equation (8).

This expectation represents the continuation value of the option at each exercise time. The main idea is that this conditional expectation can be approximated by a least square regression for each exercise time. At time t_k, it is assumed that $\mathbb{E}^Q[D(t_k, t_{k+1})V_{k+1}|\mathcal{F}_k]$ can be expressed as a linear combination of basis functions $\phi_i(x)$. That is

$$V(t_k, x) = \sum_{i=0}^{\infty} \alpha_i \phi_i(x), \quad \alpha_i \in \mathbb{R}, \tag{29}$$

which is approximated by

$$V_M(t_k, x) = \sum_{i=0}^{M} \alpha_i \phi_i(x), \quad \alpha_i \in \mathbb{R}. \tag{30}$$

Some basis functions are chosen, for example, one can use the monic polynomials. At an exercise time, the above least square regression is performed over all the in-the-money Monte Carlo paths that have positive call values to obtain the coefficients. Note that other basis functions can be used in the least square regression, such as Laguerre, Hermite, Legendre or Jacobi polynomials. Following the dynamic programming formulation Equation (28) and using the same notations as in Sections 3 and 4.1, the optimal strategy of Bermudan call option at an exercise time t_k for each Monte Carlo path $(X_k^{(j)}, Y_k^{(j)})$ can be determined by comparing the call value (i.e. the immediate exercise value) with the estimations Equation (30)

$$Y_k^{(j)}|_{t=t_k^-} = \begin{cases} Y_k^{(j)} & \text{if } V_M(t_k, X_k^{(j)}) \geq U(t_k, X_k^{(j)}) \\ U(t_k, X_k^{(j)}) & \text{if } V_M(t_k, X_k^{(j)}) < U(t_k, X_k^{(j)}) \end{cases}. \tag{31}$$

4.3. Least square backward DNN method

We summarize our least square backward DNN method as follow:

(i) M Monte Carlo (MC) paths $\{X_i^{(j)} : i = 0, 1, \ldots, N; j = 1, 2, \ldots, M; X_0^{(j)} \equiv X_0\}$ of the underlying stock are sampled by an Euler scheme through Equation (14). This step is the same as the forward DNN method.

(ii) As we have the sampled $\{X_N^{(j)} : j = 1, 2, \ldots, M\}$ available, we can calculate the payoff at expiry for the jth sampled path

$$Y_N^{(j)} = g\left(X_N^{(j)}\right). \tag{32}$$

(iii) At each time step t_i, given $\{X_i^{(j)} : j = 1, 2, \ldots, M\}$, a deep neural network (DNN) approximation is used for $\{z_i^{(j)} : j = 1, 2, \ldots, M\}$. Notice that one DNN is used for all MC paths at each time $t = t_i$. For our one dimensional problem, we introduce a DNN $\theta_i : \mathbb{R} \to \mathbb{R}$ of the form as Equation (17). $\{z_i^{(j)} = \theta_i(X_i^{(j)}) : j = 1, 2, \ldots, M\}$, and $Z_i^{(j)} = \sigma(t_i, X_i^{(j)})X_i^{(j)} z_i^{(j)}$ Then, the FBSDE propagates backward using Equation (22) in time direction from t_{i+1} to t_i. Along each Monte Carlo path, as the FBSDE propagates backward from time T to 0, $Y_0^{(j)}$ is estimated as $Y_0^{(j)}(\theta)$, where $\theta = (\theta_1, \ldots, \theta_{N-1})$ are all hyper-parameters for the $N-1$ neural networks.

(iv) During propagating from time T to 0, at exercise time t_k, the least square regression is performed and the derivative value at each path is reset using Equation (31).

(v) Set $L_{Backward}$ (Equation (24)) as the loss function.

(vi) The Adam optimization is used to minimize the loss function $L_{Backward}$ and estimate Y_0 as Equation (25). The estimated $\widetilde{Y}_0$ is our desired derivative value at $t = 0$.

Notice that the above least square backward DNN method can be easily extended to high-dimensional derivative pricing $(Y = Y(t, X^1, X^2, \ldots, X^d))$.

It is well known that the least square regression approach to exercise boundary estimation is suboptimal and produces lower biased prices. Since the least square regression is

Table 1. Market data setting.

	$r = 0.01$									
Interest Rate r	stock 1	stock 2	stock 3	stock 4	stock 5	stock 6	stock 7	stock 8	stock 9	stock 10
Spot X_0	100	150	200	175	125	100	150	200	175	125
Dividend Rate q	0.03	0.02	0.05	0.00	0.04	0.03	0.02	0.05	0.00	0.04
Volatility σ	0.2	0.3	0.25	0.24	0.15	0.2	0.3	0.25	0.24	0.15
Correlation ρ	0.3 for all ρ_{ij} when $i \neq j$									

also used in our backward DNN method, the prices produced are lower-biased, same as those from classical least square Monte Carlo (LSQ MC). Though, theoretically, the bias can be reduced with the number of regressors going to infinite, in practice, increasing the number of regressors does not always reduce bias, as observed by Moreno and Navas (2003), Stentoft (2004). Stentoft recommended second or third order polynomials to be used in regression, in particular, under high dimensions, as a trade-off between precision and computing time, and to prevent performance deterioration. Since the goal of this paper is not to assess the least square regression performance in terms of polynomials types and orders, we use second order monic polynomials as basis functions in our numerical tests (Section 5) for illustration purposes. Our testing results indicate that the monic polynomial can produce satisfactory results for the products as evidenced by consistency with the results from a finite difference PDE solver. When the least square backward DNN method is used to price early exercisable products, its accuracy will be assessed based on comparisons with the finite difference based PDE and the classical LSQ MC for up to 3 dimensions, and with the classical LSQ MC for higher than 3 dimensions.

5. Numerical results

In this section, we first use an European option to compare the performance of the forward DNN and the backward DNN methods, then use Bermudan options and CYNs as examples to test our least square backward DNN method, and compare with PDE and Monte Carlo results. Our finite difference PDE solver is only implemented for 1D, 2D and 3D cases†. For the MC solver, we use $M = 1,000,000$ sampling paths to estimate the means. Note that the relative differences between 1M and 500K are less than 0.5%.

The market data setting used in all our testing examples is described in Table 1. All of our tested examples are with $T = 1$ and time step $N = 100$ if not specified otherwise. Therefore, we have time step size $h_i = 0.01$.

The deep neural network setting in our tests is as follows: each of the neural network approximating θ_i consists of 4 layers (1 input layer [d-dimensional], 2 hidden layers [$d + 10$-dimensional], and 1 output layer [d-dimensional]), where d is the number of underlying entities. In the test, we run 5,000 optimization iterations of training and validate the trained DNN every 100 iterations. This produces 50 results.

† We implemented the 3D operator splitting method introduced by Kim *et al.* (2016).

Table 2. Comparison between forward and backward DNN on European call option.

Black Scholes	Forward DNN		Backward DNN	
NPV	NPV	rel diff to BS	NPV	rel diff to BS
6.8669	6.8688	0.03%	6.8575	− 0.14%

We use the mean of the 10 results with the least loss function value as our derivative value. The MC sampling size is $M = 5,000$.

5.1. *Forward vs. backward DNN method*

We first use a 1Y ATM single underlying stock European call option to compare the performance of the forward DNN method with the backward DNN method. We use stock 1 in Table 1 as the only underlying stock. The expiration $T = 1$ and strike $K = 100$. The results are given in Table 2 and Figure 1. Both the forward and the backward DNN methods could produce results close to the Black-Scholes price. Both methods converge fast. Small oscillations in prices can be observed in the forward DNN method. By contrast, the Backward DNN method is more stable and converges slightly faster than the forward DNN method. Therefore, the backward DNN method is the preferred approach.

5.2. *Tests on Bermudan options*

In this section, we first use 1Y ATM Bermudan options to test the performance of our least square backward DNN method. We test Bermudan call options on a single underlying stock, 2 underlying stocks (stock 1, 2), 3 underlying stocks (stock 1, 2, 3) and 5 underlying stocks (stock 1, 2, 3, 4, 5). The strike is chosen as $\sum_d \omega_i X_0^i$ with equal weight $\omega_i = 1/d$ so that the option is ATM. The Bermuda option can be exercised quarterly, or at $t = 0.25, 0.5, 0.75, 1.0$. We compare the prices from the PDE method and the Monte Carlo method with the prices from the least square backward DNN method. The results are presented in Table 3 and Figures 2 – 5. It can be seen that the backward DNN method converges very fast and the convergence rate is not sensitive to the dimensions of the problem. The results for 10, 20 and 50 dimensions are presented in Section 5.4.2 and the largest difference between the LSQ MC and the least square backward DNN method occurs at 50 dimensions with a difference of 0.4% (or 2.7 cents).

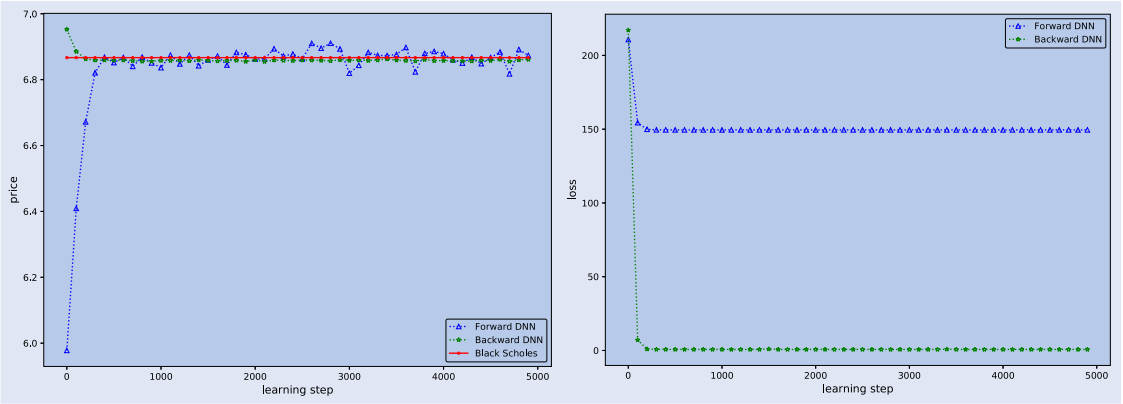

Figure 1. Comparison between forward and backward DNN on European call option.

Table 3. Comparison among PDE, Monte Carlo and least square backward DNN method on Bermudan options.

1Y ATM Bermudan Call	PDE	LSQ MC	LSQ BDNN	rel diff from PDE	rel diff from LSQ MC
single stock (stock 1)	6.9999	6.9924	6.9881	− 0.17%	− 0.06%
2 stocks (stock 1, 2)	9.9531	9.9435	9.9488	− 0.04%	0.05%
3 stocks (stock 1, 2, 3)	9.7146	9.7174	9.7203	0.06%	0.03%
5 stocks (stock 1, 2, 3, 4, 5)		8.2808	8.2795		− 0.01%

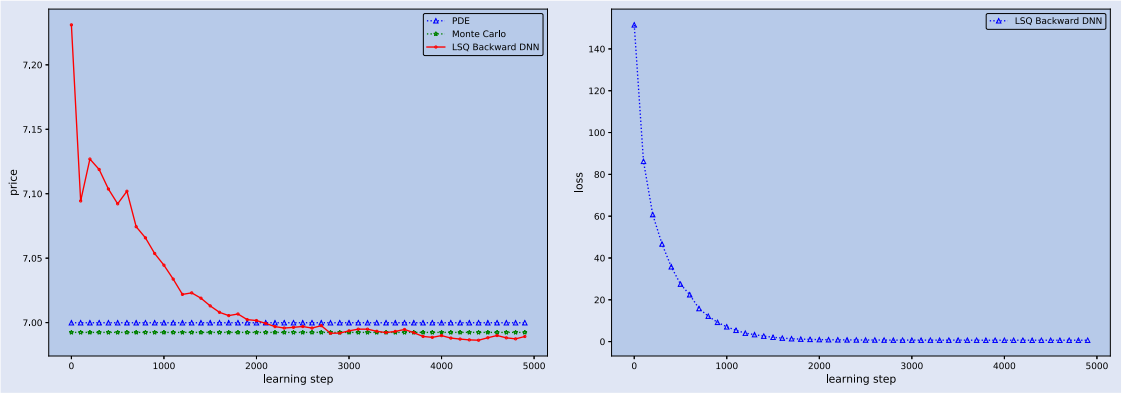

Figure 2. 1Y Bermudan Call, single underlying stock.

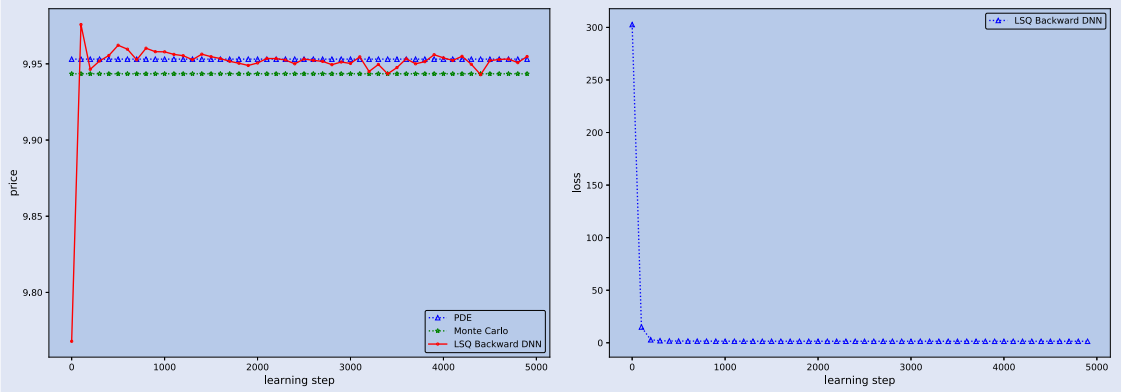

Figure 3. 1Y Bermudan Call, 2 underlying stocks.

Overall, the least square backward DNN method can produce very accurate prices for Bermudan call options.

To examine the accuracy and the stability of our least square backward DNN method under longer maturities, we also test 2Y Bermudan options. All the algorithmic parameters are the same as those introduced in the beginning of this section, except for $T = 2$ and time step $N = 240$. Therefore, we have a time step size $h_i = 1/120$. The results are presented

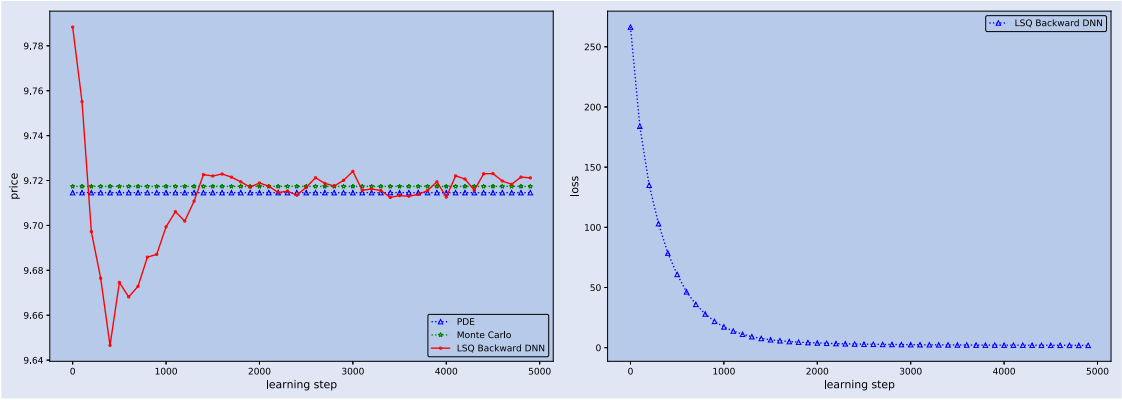

Figure 4. 1Y Bermudan Call, 3 underlying stocks.

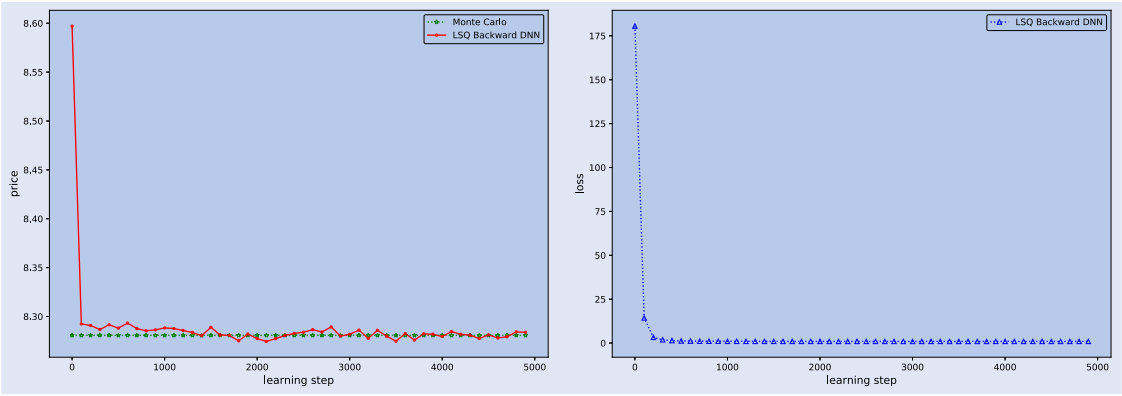

Figure 5. 1Y Bermudan Call, 5 underlying stocks.

Table 4. Comparison among PDE, Monte Carlo and least square backward DNN method on Bermudan options.

2Y ATM Bermudan Call	PDE	LSQ MC	LSQ BDNN	rel diff from PDE	rel diff from LSQ MC
single stock (stock 1)	9.3853	9.3741	9.3759	− 0.10%	0.02%
2 stocks (stock 1, 2)	13.5624	13.5298	13.5562	− 0.05%	0.20%
3 stocks (stock 1, 2, 3)	12.9047	12.8948	12.8951	− 0.07%	0.00%
5 stocks (stock 1, 2, 3, 4, 5)		11.1204	11.1112		− 0.08%

Table 5. CYN contract parameters.

contingent coupon	$r_i = 5\%$
coupon barrier	$B_i = 70\%$
knock-in barrier	$B = 50\%$
knock-in put strike	$K = 100\%$
call/coupon schedule	quarterly or 0.25, 0.5, 0.75, 1.0

in Table 4 and Figures 6 – 9. The results demonstrate both the accuracy and the stability of our backward DNN method.

5.3. Tests on callable yield notes

In this section, we use 1Y CYNs to test the performance of our least square backward DNN method for complex payoffs. We test CYNs on a single underlying stock, 2 underlying stocks (stock 1, 2), 3 underlying stocks (stock 1, 2, 3) and 5 underlying stocks (stock 1, 2, 3, 4, 5). Some key contract parameters of the tested CYNs are provided in Table 5. We compare the prices from the PDE method and the Monte Carlo method with the prices from the least square backward DNN method. The results are presented in Table 6 and Figures 10 – 13. It can be seen that all the tested samples converge fast and the differences in prices between the backward DNN approach and the PDE or MC method is very small. The results for 10, 20 and 50 dimensions are presented in Section 5.4.2. Overall, very accurate prices can be obtained with the least square backward DNN. Similar to Bermudan options, the largest difference between the LSQ MC and the least square backward DNN method occurs at 50 dimensions with a difference of 1.5% (or 1.5 cents). The results indicates the validity and accuracy of the least square backward DNN method even when the payoff is complex.

Similar to Bermudan options, we also select 2Y CYNs in testing where the time step $N = 240$ is used. The results are presented in Table 7 and Figures 14 – 17. The results indicate that our backward DNN approach has no accuracy and stability issue for early exercisable options with a complex payoff.

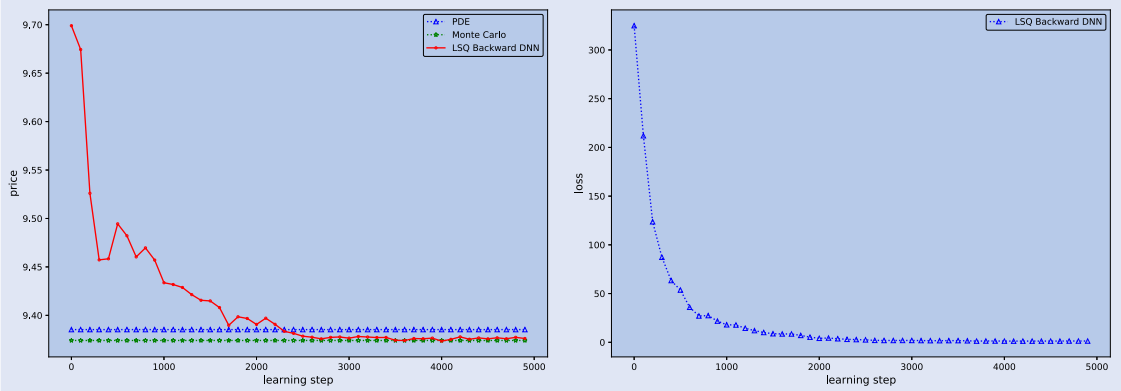

Figure 6. 2Y Bermudan Call, single underlying stock.

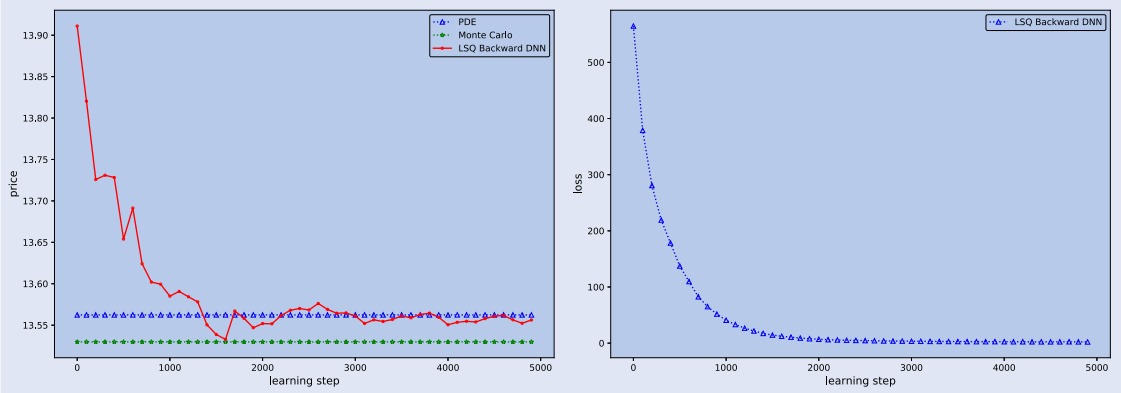

Figure 7. 2Y Bermudan Call, 2 underlying stocks.

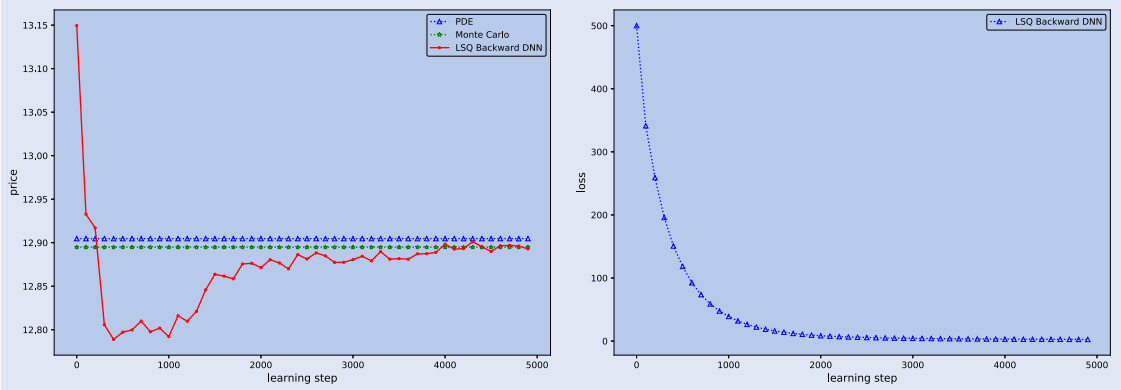

Figure 8. 2Y Bermudan Call, 3 underlying stocks.

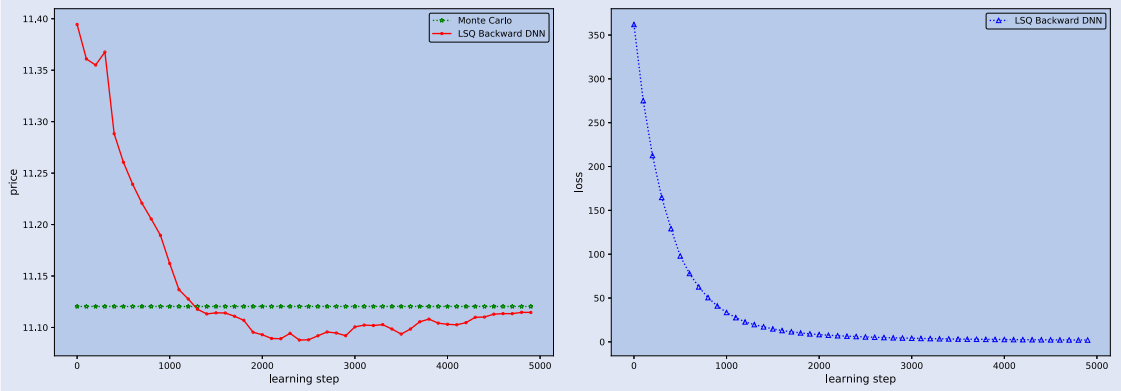

Figure 9. 2Y Bermudan Call, 5 underlying stocks.

Table 6. Comparison among PDE, Monte Carlo and least square backward DNN method on CYNs.

1Y CYN	PDE	LSQ MC	LSQ BDNN	rel diff from PDE	rel diff from LSQ MC
single stock (stock 1)	1.0474	1.0474	1.0474	0.00%	0.00%
2 stocks (stock 1, 2)	1.0456	1.0458	1.0468	0.12%	0.09%
3 stocks (stock 1, 2, 3)	1.0447	1.0452	1.0452	0.05%	0.00%
5 stocks (stock 1, 2, 3, 4, 5)		1.0450	1.0448		−0.01%

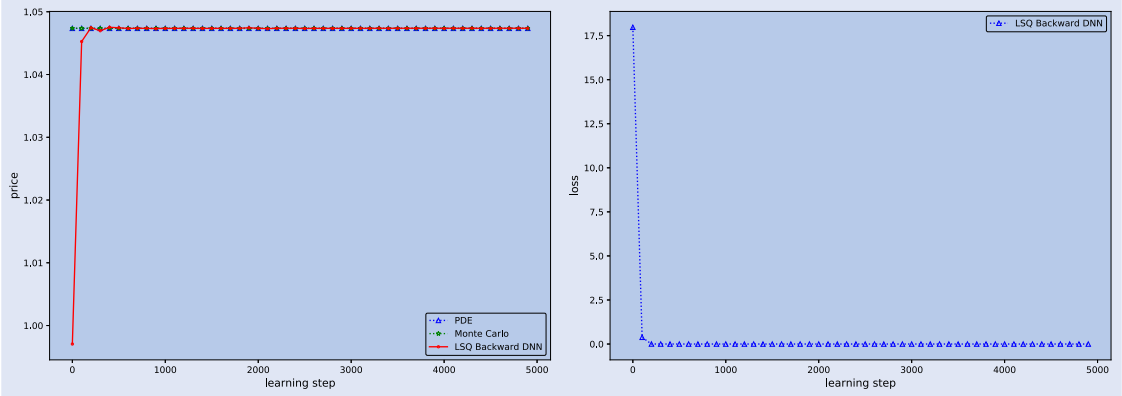

Figure 10. 1Y CYN, single underlying stock.

5.4. *Efficiency test*

In this section, we use a 1Y European option, a 1Y Bermudan option and a 1Y CYN to compare the computation efficiency between the DNN method and the classical Monte Carlo method (the least square Monte Carlo for Bermudan options and CYNs). We select 5 underlying stocks (stock 1, 2, . . . , 5), 10 underlying stocks (stock 1, 2, . . . , 10), 20 underlying stocks (repeat the 10 stocks twice) and 50 underlying stocks (repeat the 10 stocks five times) in our tests. The European/Bermudan option contract characteristics are analogous to those in Section 5.2. The strike is chosen as $\sum_d \omega_i X_0^i$ with equal weight $\omega_i = 1/d$ so that the option is ATM. The Bermuda option can be exercised quarterly, or at $t = 0.25, 0.5, 0.75, 1.0$. The CYN contract characteristics are analogous to those in Section 5.3. The same parameters are used. For the classical MC (either the regular classical MC or the least square MC) we use $M = 1,000,000$ sampling paths to estimate the means. For the least square backward DNN solver, the MC sampling size is 5,000, there are 500 optimization iterations of training, and the trained DNN is validated every 10 iterations. This produces 50 results. We use the mean(standard error) of the 10 results with the

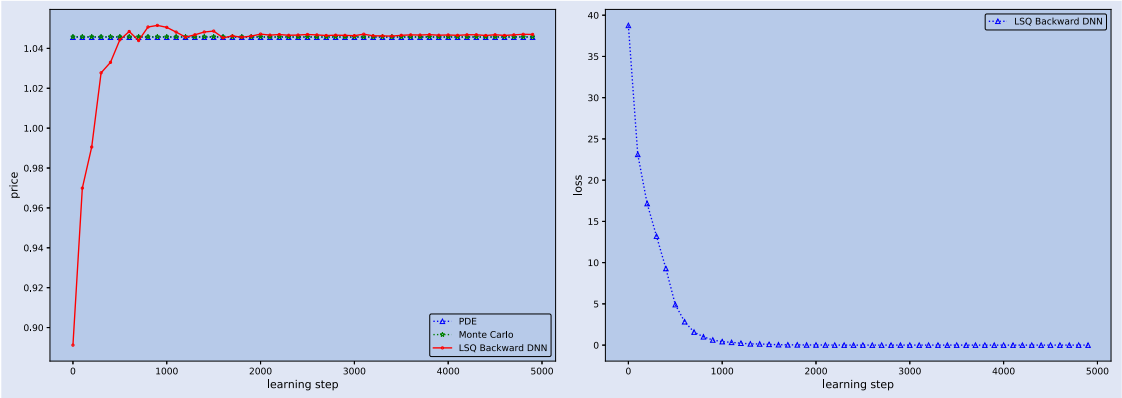

Figure 11. 1Y CYN, 2 underlying stocks.

Table 7. Comparison among PDE, Monte Carlo and least square backward DNN method on CYNs.

2Y CYN	PDE	LSQ MC	LSQ BDNN	rel diff from PDE	rel diff from LSQ MC
single stock (stock 1)	1.0473	1.0474	1.0475	0.01%	0.01%
2 stocks (stock 1, 2)	1.0418	1.0432	1.0420	0.01%	−0.12%
3 stocks (stock 1, 2, 3)	1.0344	1.0370	1.0365	0.20%	−0.05%
5 stocks (stock 1, 2, 3, 4, 5)		1.0328	1.0332		0.04%

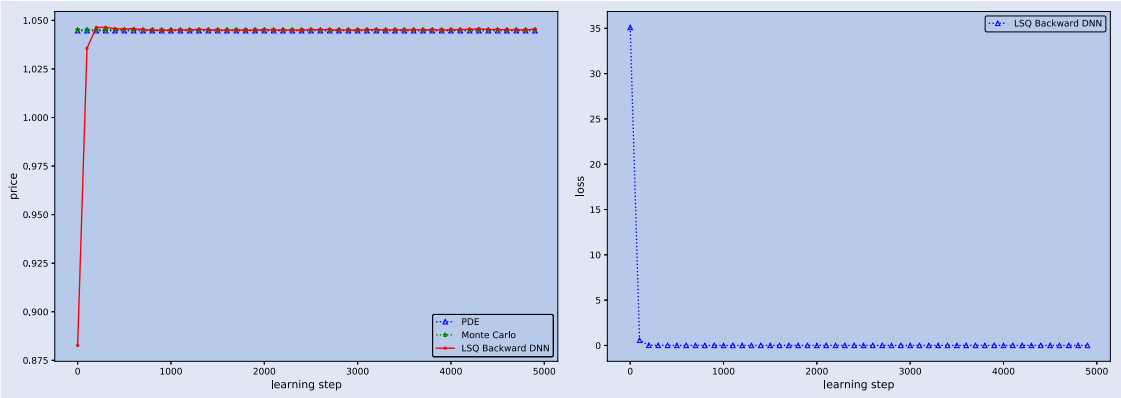

Figure 12. 1Y CYN, 3 underlying stocks.

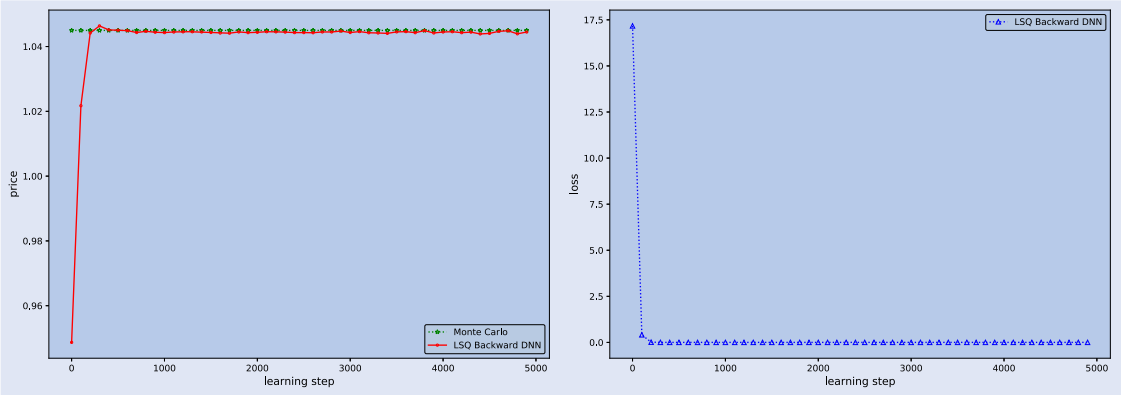

Figure 13. 1Y CYN, 5 underlying stocks.

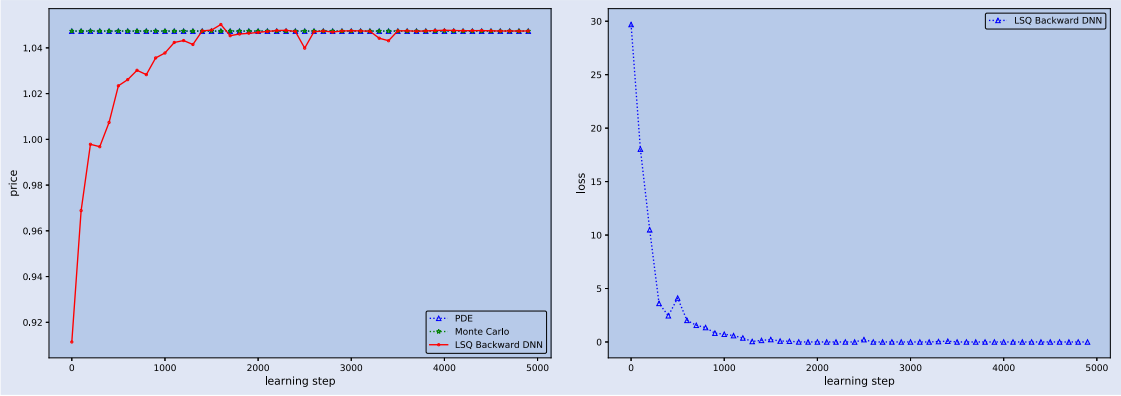

Figure 14. 2Y CYN, single underlying stock.

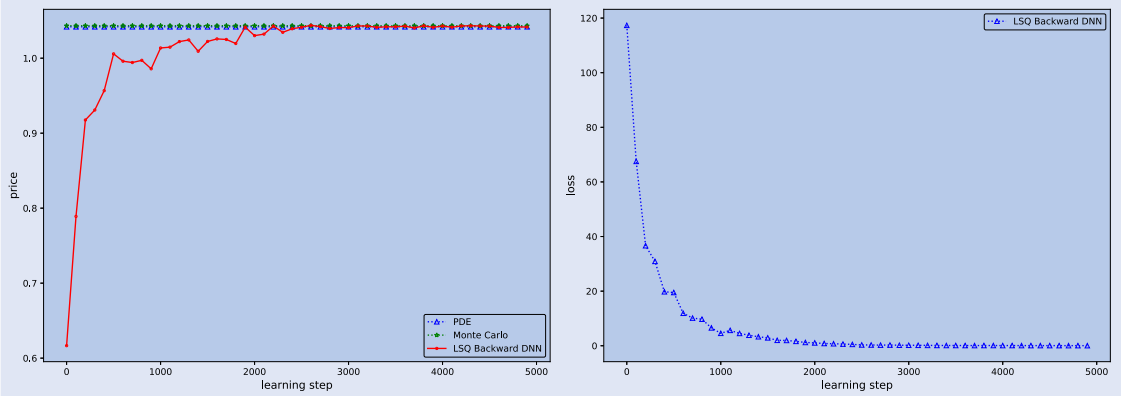

Figure 15. 2Y CYN, 2 underlying stocks.

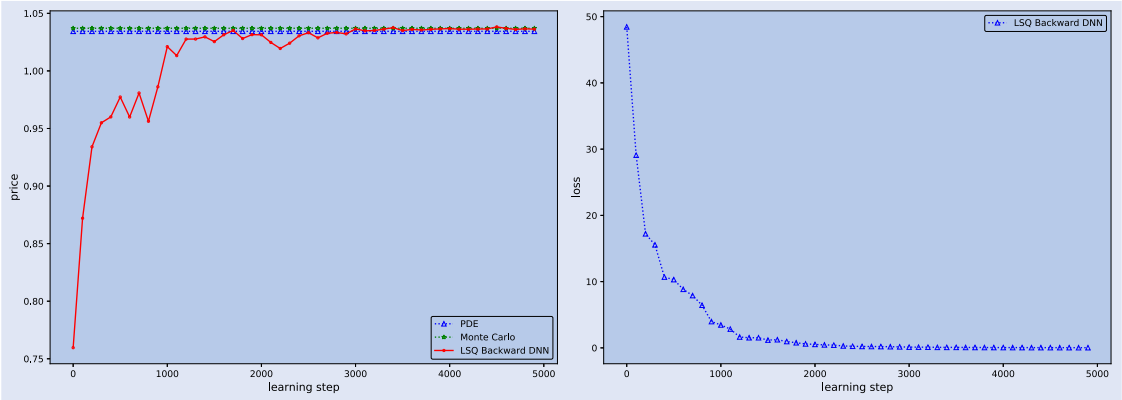

Figure 16. 2Y CYN, 3 underlying stocks.

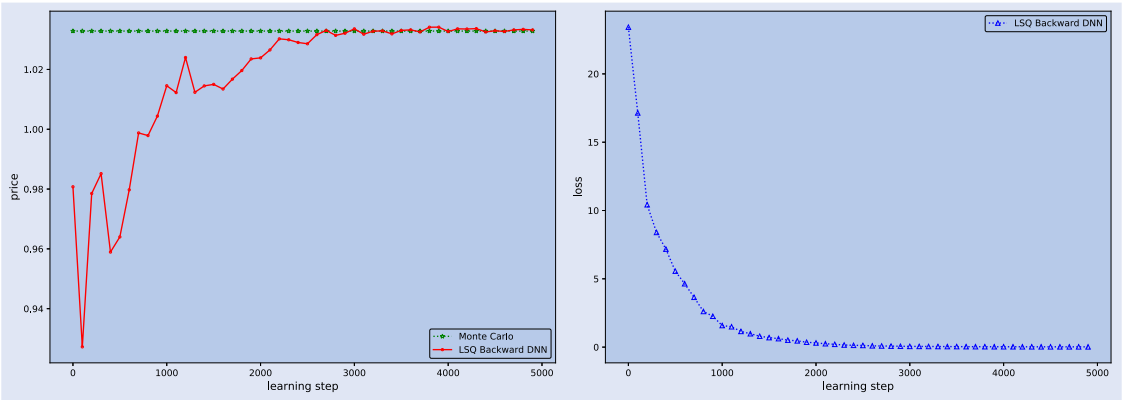

Figure 17. 2Y CYN, 5 underlying stocks.

least loss function value as our derivative price (standard error).

We performed our tests on both a desktop and a server. The testing desktop has CPU (Intel(R) Xeon(R) Silver 4108 @1.80GHz) with 8 cores/16 threads and 24GB RAM. The sever has 72 cores and 768GB RAM and each core is Intel(R) Xeon(R) E5-2699 v3 @2.30GHz.

It is well known that the parallelization of the least square Monte Carlo, widely used in high-dimensional American/Bermudan option pricing in practice, is a challenging task as the regression consumes most of the computation time for American options and Bermuda options with many early exercise times. However, since regression at each exercise time requires the cross-sectional information from all paths, regression step is not straightforward to parallelize. Realizing this characteristics, Choudhury *et al.* (2015) parallelized the singular value decomposition in the regression step. Together with path generation parallelized, an efficient ratio (speed up factor/number of processors) of 56% is achieved on a IBM Blue Gene. Chen *et al.* (2015) proposed to apply space decomposition to both the path generation phase and the regression/valuation phases. In Chen *et al.*'s work each sub-sample is an independent least square MC run. The authors found that the speed-up efficiency can be around 100% for 8 processes and around 80% for 64 processes. Even though there is a significant improvement in parallelization efficiency, pricing bias is observed and the magnitude of the bias increases with an increase in the number of processes. Therefore, instead of using sub-sampling as the parallelization strategy, we used the least square regression routine in TensorFlow to implement the regression step in the classical least-square MC after the MC samples are generated. This choice is in the same spirit as the approach from Choudhury et al., i.e. parallelizing the most time consuming component of the process. Since the classical least-square MC is our benchmark to assess the accuracy of our backward DNN method, our approach avoids potential sub-sampling bias in the results from classical least-square MC. In addition, since the computational resources are fully managed by TensorFlow for both the classical least-square MC and the backward DNN, we have a fair comparison of efficiency for the two approaches.

5.4.1. Efficiency tests on European options. The testing results for a European option from 5 dimensions to 50 dimensions are presented in Table 8. OOM means 'out-of-memory'. It can seen that the backward DNN method can produce prices very close (around 2 cents or less) to those from the classical MC. Though the classical MC is faster, it can not produce results for 20 dimensions and above with a desktop due to memory issues. The backward DNN method is slower, but results can be produced with all the cases we tested since it does not need a large number of samples to produce accurate results. Apparently, in general, if one has sufficient computational resource, DNN method is not efficient to price European style options as it is 5 to 6 times slower than the classical MC, primarily caused by the high cost in the DNN initialization and optimization. However, if one only has a

Table 8. Comparison between Monte Carlo and backward DNN method on European options.

1Y ATM European Call	MC				BDNN			
	price	std error	desktop(s)	server(s)	price	std error	desktop(s)	server(s)
5 stocks	8.1033	0.0142	78	57	8.1146	0.0169	486	330
10 stocks	7.2546	0.0127	157	112	7.2318	0.0159	882	544
20 stocks	6.8038	0.0119	OOM	230	6.7856	0.0144	1904	981
50 stocks	6.5121	0.0113	OOM	576	6.4975	0.0137	6290	2650

Table 9. Comparison between Monte Carlo and least square backward DNN method on Bermundan options.

1Y ATM Bermudan Call	LSQ MC				LSQ BDNN			
	price	std error	desktop(s)	server(s)	price	std error	desktop(s)	server(s)
5 stocks	8.2709	0.0124	82	60	8.2795	0.0160	509	345
10 stocks	7.4112	0.0110	177	116	7.4127	0.0153	972	569
20 stocks	6.9760	0.0103	OOM	237	6.9745	0.0141	2549	976
50 stocks	6.7372	0.0100	OOM	658	6.7100	0.0137	21552	3476

Table 10. Comparison between Monte Carlo and least square backward DNN method on CYNs.

1Y CYN	LSQ MC				LSQ BDNN			
	price	std error	desktop(s)	server(s)	price	std error	desktop(s)	server(s)
5 stocks	1.0449	0.0000	84	60	1.0448	0.0008	506	342
10 stocks	1.0402	0.0001	179	122	1.0390	0.0011	947	563
20 stocks	1.0257	0.0001	OOM	235	1.0236	0.0016	2860	1012
50 stocks	0.9778	0.0002	OOM	659	0.9633	0.0023	20947	3274

machine with limited memory, the DNN approach is a choice for large scale problems.

5.4.2. Efficiency tests on Bermudan options and CYNs. The testing results from 5 dimensions to 50 dimensions are presented in Table 9 for Bermudan options and in Table 10 for CYNs. First, it can be seen that the backward DNN method (here the least square backward DNN) can produce prices very close (around 1 cent or less in most cases) to those from the classical least square MC, an evidence of its validity and accuracy. It is also very interesting to note that for Bermudan options, the standard deviations of the results from the backward DNN are very similar to the MC errors of the mean from the classical MC approach with our choice of parameters. For CYNs, though the standard deviations of the results from the backward DNN are one order higher than the MC errors of the mean from the classical MC approach, they are still sufficiently small. Similar to the efficient tests for European options, the classical MC approach, though 5 or more times faster, could not produce results for 20 dimensions and above with a desktop computer due to memory issues.

6. Conclusion

In this work, we have developed a deep learning-based least squares forward-backward stochastic differential equation solver, which can be used for high-dimensional derivatives pricing. Our deep learning implementation follows a similar approach to the ones explored by E *et al.* (2017), Han *et al.* (2018), and Wang *et al.* (2018). However, the forward DNN method is more suitable for European style derivative pricing and the backward DNN method from Wang *et al.* may not adequately account for early exercise features. In our approach, we embed a least squares regression technique, similar to that in the least squares Monte Carlo method (Longstaff and Schwartz 2001) in the backward DNN algorithm. Numerical test results on Bermudan options and callable yield notes indicate that our least squares backward DNN method can produce very accurate results relative to the finite difference based PDE approach or of classical Monte Carlo simulation. This method can be used in various derivative pricing applications such as for Barrier options, American options, convertible bonds, etc. In conclusion, our least squares backward DNN algorithm can serve as a generic numerical solver for pricing derivatives, and it is most suitable for high-dimensional derivatives with early exercise features. Though it is slower than classical Monte Carlo, the backward DNN approach is very memory efficient and can handle large scale problems with fewer computational resources.

Acknowledgements

The authors would like to thank Professor Jim Gatheral for his valuable comments and suggestions for a major revision of this paper and to two anonymous referees for their careful reading and numerous suggestions. The authors also would like to thank Dr. Agus Sudjianto for introducing them to the field of using machine learning in derivative pricing and thank Prof. Liuren Wu for reviewing the manuscript. The authors

also gratefully acknowledge Dr. Bernhard Hientzsh for highly valuable discussions.

Disclosure statement

No potential conflict of interest was reported by the author(s).

References

Barraquand, J. and Martineau, D., Numerical valuation of high dimensional multivariate American securities. *J. Financ. Quant. Anal.*, 1995, **30**(3), 383–405.

Becker, S., Cheridito, P. and Jentzen, A., Deep optimal stopping. *J. Mach. Learn. Res.*, 2019, **20**(74), 1–25.

Becker, S., Cheridito, P., Jentzen, A. and Welti, T., Solving high-dimensional optimal stopping problems using deep learning. arXiv preprint arXiv:1908.01602. 2019.

Bouchard, B. and Chassagneux, J., Discrete-time approximation for continuously and discretely reflected BSDEs. *Stoch. Process. Their. Appl.*, 2008, **118**(12), 2269–2293.

Broadie, M. and Glasserman, P., Pricing American-style securities using simulation. *J. Econ. Dynam. Control*, 1997, **21**(8–9), 1323–1352.

Chen, C., Huang, K. and Lyuu, Y., Accelerating the least-square monte carlo method with parallel computing. *J. Supercomput.*, 2015, **71**(9), 3593–3608.

Choudhury, A.R., King, A., Kumar, S. and Sabharwal, Y., Optimizations in financial engineering: The least-squares Monte Carlo method of Longstaff and Schwartz. In *2008 IEEE International Symposium on Parallel and Distributed Processing*, pp. 1–11, 2015 (IEEE: Miami, FL).

Clément, E., Lamberton, D. and Protter, P., An analysis of a least squares regression method for American option pricing. *Finance Stoch.*, 2002, **6**, 449–471.

De Spiegeleer, J., Madan, D.B., Reyners, S. and Schoutens, W., Machine learning for quantitative finance: Fast derivative pricing, hedging and fitting. *Quant. Finance*, 2018, **18**(10), 1635–1643.

E, W., Han, J. and Jentzen, A., Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Commun. Math. Statist.*, 2017, **5**(4), 349–380.

El Karoui, N., Kapoudjian, C., Pardoux, E., Peng, S. and Quenez, M., Reflected solutions of backward SDE'S, and related obstacle problems for PDE'S. *Ann. Probab.*, 1997, **25**(2), 702–737.

El Karoui, N., Peng, S. and Quenez, M., Backward stochastic differential equations in finance. *Math. Finance*, 1997, **7**(1), 1–71.

El Karoui, N., Pardoux, E. and Quenez, M., Reflected backward SDEs and American options. *Numer. Methods Finance*, 1997215–231.

Fujii, M., Takahashi, A. and Takahashi, M., Asymptotic expansion as prior knowledge in deep learning method for high dimensional BSDEs. *Asia-Pacific Financ. Markets*, 2019, **26**(3), 391–408.

Han, J., Jentzen, A. and E, W., Solving high-dimensional partial differential equations using deep learning. *Proc. National Acad. Sci.*, 2018, **115**(34), 8505–8510.

Hull, J.C., Chapter 1 Introduction. In *Options, Futures, and Other Derivatives*, 10th ed., 2018 (Pearson: London).

Huré, C., Pham, H. and Warin, X., Deep backward schemes for high-dimensional nonlinear PDEs. *Math. Comput.*, 2020, **89**(324), 1547–1579.

Hutchinson, J.M., Lo, A.W. and Poggio, T., A nonparametric approach to pricing and hedging derivative securities via learning networks. *J. Finance.*, 1994, **49**(3), 851–889.

Kim, J., Kim, T., Jo, J., Choi, Y., Lee, S., Hwang, H., Yoo, M. and Jeong, D., A practical finite difference method for the three-dimensional Black–Scholes equation. *Eur. J. Oper. Res.*, 2016, **252**(1), 183–190.

Kohler, M., Krzyżak, A. and Todorovic, N., Pricing of high-dimensional American options by neural networks. *Math. Finance*, 2010, **20**(3), 383–410.

Longstaff, F. and Schwartz, E., Valuing American options by simulation: A simple least-squares approach. *Rev. Financ. Stud.*, 2001, **14**(1), 113–147.

Moreno, M. and Navas, J.F., On the robustness of least-squares Monte Carlo (LSM) for pricing American derivatives. *Rev. Derivatives Res.*, 2003, **6**, 107–128.

Mil'shtejn, G., Approximate integration of stochastic differential equations. *Theory Probab. Appl.*, 1975, **19**(3), 557–562.

Neveu, J., *Discrete-parameter Martingales*, 1975 (North Holland: Amsterdam).

Raissi, M., Forward-backward stochastic neural networks: Deep learning of high-dimensional partial differential equations. arXiv preprint arXiv:1804.07010, 2018.

Sirignano, J. and Spiliopoulos, K., Stochastic gradient descent in continuous time. *SIAM J. Financ. Math.*, 2017, **8**(1), 933–961.

Sirignano, J. and Spiliopoulos, K., DGM: A deep learning algorithm for solving partial differential equations. *J. Comput. Phys.*, 2018, **375**(15), 1339–1364.

Stentoft, L., Assessing the least squares Monte-Carlo approach to American option valuation. *Rev. Derivatives Res.*, 2004, **7**(2), 129–168.

Wang, H., Chen, H., Sudjianto, A., Liu, R. and Shen, Q., Deep learning-based BSDE solver for Libor market model with applications to Bermudan swaption pricing and hedging. arXiv preprint arXiv:1807.06622, 2018.